SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA0301	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	50% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	B.Jahnavi	**Reg. No.:**	192325043
Section / Batch:	4^{th} year / AIML	**Date:**	07/08/26

Code of Conduct

I B.Jahnavi / 192325043 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: B.Jahnavi

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3
Finite-State Morphological Analysis, Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

Annexure A – ANALYTICAL PROBLEM SOLVING

1. Given the input words: "connected", "connecting", "connection", design a morphological analysis pipeline for a document indexing system.
Apply rules to decompose each word into root and suffix components.
Differentiate between inflectional and derivational forms within the dataset.
Normalize all variants to a common base form for efficient retrieval.
Determine the parsed structure and normalized output for each word.

ANSWER :

A morphological analysis pipeline for a document indexing system aims to identify the base form (lemma) of words while preserving useful grammatical information.

Morphological Analysis Pipeline

1. Input Tokenization
   - Read individual words from the document.
   - Example input:
     - connected
     - connecting
     - connection
2. Morphological Parsing
   - Identify the root (stem) and suffix using morphological rules.
3. Morphological Classification
   - Determine whether the suffix represents:
     - Inflectional morphology (changes grammatical form but not meaning).
     - Derivational morphology (creates a new word or changes part of speech).
4. Lemmatization (Normalization)
   - Convert all variants to a common base form for indexing.
   - Store the normalized form while optionally retaining the original word.

MORPHOLOGICAL RULES :

Rules	Description	Type
Root+ -ed	Past tense / past participle	Inflectional
Root + -ing	Present participle / gernund	Inflectional
Root + -ion	Forms a noun from a web	Inflectional

ANALYSIS OF EACH WORD :

Input word	Root	Suffix	Morphological Type	Parsed Structure	Normalized form
connected	connect	-ed	Inflectional	connect+ed	connect
connecting	connect	-ing	Infectional	connect+ing	connect
connection	connect	-ion	Derivational	connect+ion	connect

Explanation

1. connected

Root: connect
Suffix: -ed
Indicates past tense or past participle.
Since the meaning remains the same, this is inflectional.

2. connecting
Root: connect
Suffix: -ing
Indicates present participle or gerund.
This is also inflectional.

3. connection
Root: connect
Suffix: -ion
Converts the verb *connect* into the noun *connection*.
This changes the grammatical category, making it derivational.

Final Parsed Structures
connected → connect + ed (Inflectional) → connect
connecting → connect + ing (Inflectional) → connect
connection → connect + ion (Derivational) → connect

2.Given the input words: "unhappy", "happiness", "happily", design a morphological parsing module for sentiment analysis.
Identify prefixes, suffixes, and base forms using rule-based decomposition.
Ensure semantic consistency across derived word forms.
Classify each transformation as inflectional or derivational.
Produce the root and morphological breakdown for each word.

ANSWER:

Morphological Parsing Module for Sentiment Analysis

The purpose of the module is to decompose words into their morphological components while preserving the core sentiment meaning. All variants are mapped to the common base form **happy**.

Rule-Based Morphological Parsing Pipeline

1. **Input Processing**
    1. Read each input word.
    2. Example: *unhappy, happiness, happily*
2. **Affix Identification**
    1. Detect **prefixes** (e.g., *un-*).
    2. Detect **suffixes** (e.g., *-ness*, *-ly*).
3. **Root Extraction**
    1. Remove recognized affixes using predefined rules.
    2. Recover the base/root word.
4. **Morphological Classification**
    1. Determine whether the transformation is:
        1. **Inflectional** – changes grammatical form only.
        2. **Derivational** – creates a new word or changes meaning/part of speech.

5.Normalization

Map all derived forms to the base form **happy** for consistent sentiment analysis.

Rule-Based Decomposition

Rule	Description	Type
un- + adjective	Creates the opposite meaning	Derivational
adjective + -ness	Forms a noun	Derivational
adjective + -ly	Forms an adverb	Derivational

Morphological Analysis

Input	Prefix	Root	Suffix	Morphological breakdown	Transformation type	Normalized form
unhappy	un--	happy	--	un+happy	Derivational	happy
happiness	--	happy	-ness	happy+ness	Derivational	happy
happily	--	happy	-ly	happy+ly	Derivational	happy

Semantic Consistency

- **happy** → Positive emotion (base form).
- **unhappy** → Opposite (negative sentiment); prefix **un-** reverses the meaning.
- **happiness** → Refers to the state or quality of being happy; retains the positive sentiment concept.
- **happily** → Describes an action performed in a happy manner; retains the positive sentiment.

For sentiment analysis:

- The base concept is **happy**.
- The **un-** prefix indicates **negative polarity**.
- The suffixes **-ness** and **-ly** do not reverse sentiment; they only change the grammatical category.

Final Morphological Breakdown

- **unhappy** → **un + happy** → Prefix: **un-**, Root: **happy** → **Derivational**
- **happiness** → **happy + ness** → Root: **happy**, Suffix: **-ness** → **Derivational**
- **happily** → **happy + ly** → Root: **happy**, Suffix: **-ly** → **Derivational**

Normalized Output

Original Word	Normalized Base Form
unhappy	happy
happiness	happy
happily	happy

3. Given the input words: “played”, “player”, “playing”, design a stemming-based preprocessing module for a search engine.
Apply morphological rules to strip affixes and identify the base form.
Handle both grammatical variations and word formation changes.
Ensure consistent root extraction across all forms.
Determine the stem and classification for each input word.

ANSWER:

Stemming-Based Preprocessing Module for a Search Engine

A **stemming** module removes common prefixes or suffixes from words to obtain a common **stem**, allowing different word forms to match the same search query.

Stemming Pipeline

1. **Input Tokenization**
   1. Read the input words.
   2. Example: played, player, playing
2. **Affix Stripping**
   1. Apply rule-based stemming to remove common suffixes:
      1. **-ed** → Past tense
      2. **-ing** → Present participle/Gerund
      3. **-er** → Agent noun
3. **Stem Extraction**
   1. Identify the common stem after removing the suffix.
4. **Classification**
   1. **Inflectional:** Changes only grammatical form.
   2. **Derivational:** Creates a new word or changes the part of speech.
5. **Normalization**
   1. Index all variants using the common stem **play** for efficient retrieval.

Morphological Rules

Rule	Description	Type
Root + -ed	Past tense	Inflectional
Root + -ing	Present participle/Gerund	Inflectional
Root + -er	Forms a noun meaning "one who performs the action"	Inflectional

Morphological Analysis

Input	stem	Suffix removed	Morphological breakdown	Classification	Normalized form
played	play	-ed	Play+ed	Inflectional	play
player	play	-er	Play+er	Derivational	play
playing	play	-ing	Play+ing	Inflectional	play

Explanation
1. played

- Stem: play
- Suffix: -ed
- Breakdown: play + ed
- Classification: Inflectional (past tense)

2. player

- Stem: play
- Suffix: -er
- Breakdown: play + er
- Classification: Derivational (forms a noun meaning "a person who plays")

3. playing

- Stem: play
- Suffix: -ing
- Breakdown: play + ing
- Classification: Inflectional (present participle/gerund)

Final Stemming Output:

Original word	Stem	Classification
played	play	Inflectional
player	play	Derivational
playing	play	Inflectional

4. Given the input words: “writes”, “writing”, “written”, design a finite-state morphological parsing module for a text processing system. Apply state transitions to represent suffix transformations and irregular forms in verb inflection. Differentiate between regular and irregular inflectional patterns within the dataset. Ensure consistent mapping of all forms to a common root representation. Determine the state transitions, morphological breakdown, and normalized output for each word.

ANSWER:

Finite-State Morphological Parsing Module for a Text Processing System

A Finite-State Morphological Parser (FSM) recognizes word forms by moving through states according to suffixes and irregular forms, then maps every form to the common root write.

Finite-State Parsing Model

States

- S0 – Start state
- S1 – Root recognized (write)
- S2 – Regular inflection (-s)
- S3 – Regular inflection (-ing)
- S4 – Irregular past participle (written)
- SF – Final/Accept state

For this morphological parser, the transitions are:

Current state	Input pattern	Next state	Output
S0	write	S1	Root identified
S1	+s	S2	writes
S1	+ing	S3	writing
S1	Irregular form written	S4	written
S2,S3,S4	Accept	SF	Normalized

Morphological Rules

Rule	Description	Pattern
Root + -s	Third-person singular present	Regular inflection
Root + -ing	Present participle/Gerund	Regular inflection
Write -> written	Past participle with vowel change + -en	Irregular inflection

Morphological Analysis

Input word	Morphological Breakdown	State Transition	Pattern	Normalized output
writes	Write + s	S0 → S1 → S2 → SF	Regular inflection	write
writing	Write+ing	S0 → S1 → S3 → SF	Regular inflection	write
written	Write-> written	S0 → S4 → SF	Irregular inflection	write

Regular vs. Irregular Inflection

Word	Type	Reason
writes	Regular inflection	Adds **-s** for third-person singular
writing	Regular inflection	Adds -ing for present participle
written	Irregular inflection	Internal vowel change plus **-en**, not formed by simple suffix addition

Final Morphological Breakdown

- **writes** → **write + s** → Regular inflection → **write**
- **writing** → **write + ing** → Regular inflection → **write**
- **written** → **write** → **written** (irregular past participle) → Irregular inflection → **write**

Normalized Output

Original word	Root	Classification	Normalized form
writes	write	Regular inflection	write
writing	write	Regular inflection	write
written	write	Irregular inflection	write

5. Given the input words: “relational”, “relation”, “relate”, design a Porter Stemmer-based preprocessing module for document retrieval.
Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.
Handle variations in suffix patterns while preserving semantic consistency.
Ensure uniform root extraction across all derived forms.
Determine the stemming steps, intermediate forms, and final stem for each input word.

A Porter Stemmer-based preprocessing module removes common suffixes from words to obtain a common stem. This allows related word forms to be indexed together and improves document retrieval.

Porter Stemming Rules

The Porter Stemmer applies suffix-removal rules in stages. For these words, the important rules are:

-al → ∅ : removes the suffix -al.
-ion → ∅ : removes -ion when the remaining stem satisfies the required conditions.
-e → ∅ : removes a final -e under the appropriate conditions.

Step-by-Step Stemming

Input word	Stemming steps	Intermediate form	Final stem
relational	relational → relate (remove -ion after applying the -ational → -ate rule) → relat (remove final -e)	relate → relat	relat
relation	relation → relate (remove -ion using -ion → -e rule) → relat (remove final -e)	relate → relat	relat
relate	relate → relat (remove final -e)	relat	relat

Detailed Analysis

1. relational

Input: relational
Apply the Porter rule ational → ate:
relational → relate
Apply the final e removal rule:
relate → relat
Final stem: relat

2. relation

Input: relation
Apply the Porter ion transformation:
relation → relate
Apply the final e removal rule:
relate → relat
Final stem: relat

3. relate

Input: relate
Apply the final e removal rule:
relate → relat
Final stem: relat

Final Normalized Output

Original word	Final stem
relational	relat
relation	relat
relate	relat

Rubrics for Calculation

Numerical Problems					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q. No. / Criteria Level	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: ________________	Signature: ________________	Signature: ________________
Date: ____________________	Date: ____________________	Date: ____________________